DD_EM-03 - CVM Retention And Recovery - Developer Implementation Guide

Version: v1.0 **Bundle:** 10_other **Companion DD:** DD_EM-03-CVM_Retention_and_Recovery-v1.0

1. Goal

Implement CVM as an event-driven customer activity and offer orchestration module.

EM-03 consumes signals, evaluates rules, creates activities/offers, calls owning modules, and records outcomes. It does not own billing, subscription, discount runtime, campaigns, notifications, or customer master.

2. Runtime Flow

```
sequenceDiagram
  participant EVT as BIL/SUB/TCK Events
  participant CVM as EM-03
  participant ILM as ILM-CFG-01
  participant RULES as Drools
  participant SIP as SIP-03/SIP-05
  participant NOT as NOT-01
  participant INT as CUST-INT
  participant APR as EM-CFG-04

  EVT->>CVM: Dunning/Ticket/Subscription event
  CVM->>ILM: read customer/contact summary
  CVM->>CVM: update signal profile
  CVM->>RULES: evaluate segment/activity/offer
  CVM->>APR: approval if offer exceeds policy
  CVM->>SIP: apply accepted discount/campaign action
  CVM->>NOT: request customer notification
  CVM->>INT: record activity/outcome
```

3. Step 1 - Migrations

Create:

- `cvm_customer_signal_profile`
- `cvm_segment_membership`
- `cvm_activity`
- `cvm_offer_instance`
- `cvm_outcome`

Add unique/idempotency support:

- unique `(operator_code, customer_id)` on signal profile;
- unique source-event idempotency for automatically created activities;
- indexes by `assigned_to_user_id`, `segment_code`, and `status`.

4. Step 2 - Consume Signals

Consume events such as:

- `SubscriptionDunningLevelChanged`
- `SubscriptionDunningCleared`
- `PaymentApplied`
- `TicketCreated`

- `ComplaintCreated`
- `SubscriptionDowngraded`
- `SubscriptionCancelled`
- `SubscriptionRestrictionAdded`
- `WorkOrderFinalized`

For each event:

1. Normalize customer/account/subscription ids.
2. Load current customer summary from ILM if needed.
3. Upsert `cvm_customer_signal_profile`.
4. Call evaluation.

5. Step 3 - Evaluate Customer

Endpoint:

```
POST /api/cvm/customers/CUS-100045/evaluate
Content-Type: application/json
```

Request:

```
{
  "operatorCode": "WIK",
  "reason": "DUNNING_EVENT",
  "sourceEventRef": "dunning-evt-001"
}
```

Run Drools:

```
public record CvmCustomerFact(
    String operatorCode,
    String customerId,
    int dunningLevel,
    int openTicketCount,
    int complaintCount90d,
    int daysSinceLastPayment,
    double churnRiskScore,
    double upsellScore,
    boolean hasActiveRetentionActivity
) {}
```

```
package rules.cvm.segmentation

rule "High risk recovery customer"
when
  $f : CvmCustomerFact(dunningLevel >= 2,
                       complaintCount90d >= 2,
                       hasActiveRetentionActivity == false)
  $d : CvmDecision()
then
  $d.addSegment("RETENTION_HIGH_RISK");
  $d.createActivity("PAYMENT_RECOVERY", "HIGH");
end
```

Store segment membership and create activities from the decision.

6. Step 4 - Create Activity

Endpoint:

```
POST /api/cvm/activities
Idempotency-Key: cvm-dunning-evt-001
Content-Type: application/json
```

Request:

```
{
  "operatorCode": "WIK",
  "activityType": "PAYMENT_RECOVERY",
  "customerId": "CUS-100045",
```

```
  "accountId": "ACC-900045",
  "subscriptionId": "SUB-700045",
  "sourceEventRef": "dunning-evt-001",
  "assignedTeamId": "team-retention",
  "priority": "HIGH"
}
```

After insert:

1. Emit `CvmActivityCreated`.
2. Write `CUST-INT` interaction of type `CVM_ACTIVITY_CREATED`.
3. Optionally request NOT-01 notification.

7. Step 5 - Propose Offer

Endpoint:

```
POST /api/cvm/offers
Content-Type: application/json
```

Request:

```
{
  "operatorCode": "WIK",
  "activityId": "cva-001",
  "customerId": "CUS-100045",
  "subscriptionId": "SUB-700045",
  "offerType": "RETENTION_DISCOUNT",
  "campaignCode": "RET-10PCT-3M",
  "discountRef": "DISC-RET-10"
}
```

Run offer rules:

```
package rules.cvm.offer

rule "High value retention discount requires approval"
when
  $f : CvmOfferFact(offerType == "RETENTION_DISCOUNT",
                    discountPercent > 10)
  $d : CvmDecision()
then
  $d.requireApproval("CVM_HIGH_VALUE_RETENTION_OFFER");
end
```

If approval required, call EM-CFG-04 and keep offer `DRAFT` or `PENDING_APPROVAL`.

Camunda is used only when the selected EM-CFG-04 approval policy mode is `BPMN`.

8. Step 6 - Accept And Apply Offer

Endpoint:

```
POST /api/cvm/offers/cvo-001/accept
Content-Type: application/json

{
  "acceptedByUserId": "usr-retention-01",
  "customerConsentRef": "consent-001"
}
```

If offer type is `RETENTION_DISCOUNT`, call SIP-03:

```
POST /api/discount-assignments
Idempotency-Key: cvo-001-discount
Content-Type: application/json

{
  "operatorCode": "WIK",
  "customerId": "CUS-100045",
  "subscriptionId": "SUB-700045",
  "discountRef": "DISC-RET-10",
  "campaignCode": "RET-10PCT-3M",
  "originRefType": "CVM_OFFER",
  "originRefId": "cvo-001"
}
```

For upgrade offers, call SUB-WF upgrade preview/request APIs instead.

9. Step 7 - Close Activity

When outcome is known:

1. Insert `cvm_outcome`.
2. Update offer/activity status.
3. Emit event.
4. Write CUST-INT interaction.
5. Let REP-01 consume events for dashboards.

10. Tests

1. Dunning event upserts signal profile.
2. Drools creates high-risk segment and activity.
3. Duplicate source event does not create duplicate activity.
4. Offer above approval threshold calls EM-CFG-04.
5. Accepted discount calls SIP-03, not DIS-OP directly.
6. NOT-01 is used for customer notifications.
7. CUST-INT receives activity and outcome records.